

RustでOAuth2+Passkeyのライブラリを作ってます

高橋 公俊 (Kimitoshi Takahashi)

dev_night Tokyo #5 | 2026/04/23

Rustと、認証認可の修行のために
oauth2-passkey というライブラリを作ってます

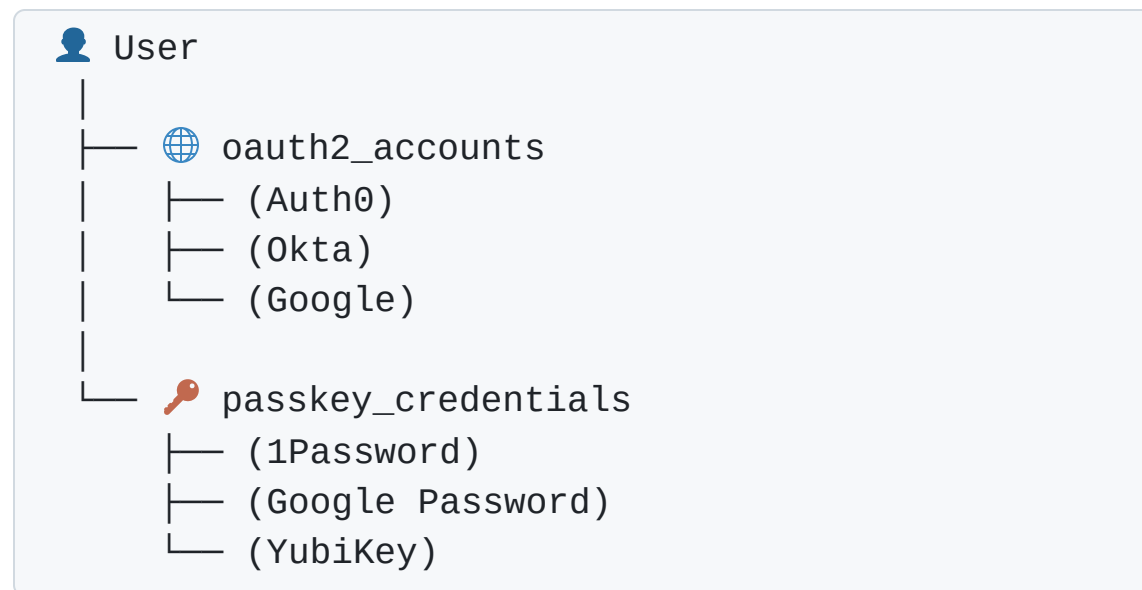
→ 今日のLTのために **Auth0 / Okta**
使えるようにしました

...やっと今日動きました

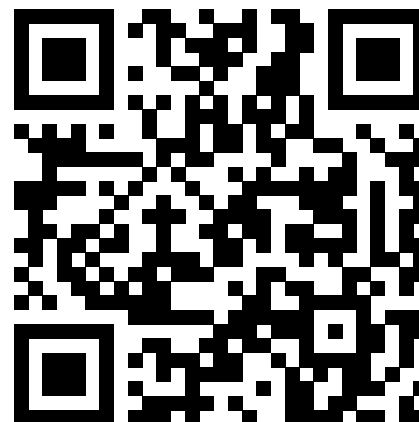
oauth2-passkeyとは

OAuth2 / Passkey で認証 → Session Cookie 発行
crates.io 公開済み: `oauth2-passkey{, -axum}`

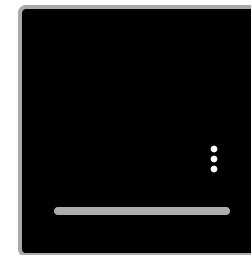
複数のIdP/Passkey を一ユーザーに紐付け



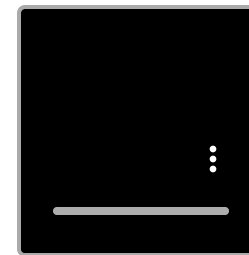
1. Auth0 / Okta でユーザー作成
2. **Passkey** を登録
3. 次回から Passkey だけでログイン



デモサイト



デモ動画



デモ動画

3 行で組み込める

```
use oauth2_passkey_axum::{AuthUser, oauth2_passkey_full_router}; // 1. インポート

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?; // 2. 初期化

    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 3. ルータ合流

    spawn_http_server(3001, app).await?;
    Ok(())
}
```

- OAuth2(OIDC)/Passkey のエンドポイント、ログインUI、管理画面 が自動作成される
- ページ保護は `AuthUser` extractor または middleware で

対応 OAuth2 / OIDC プロバイダー

Auth0

Okta

Google

Microsoft Entra

Keycloak

Zitadel

Authentik

Custom (OIDC)

.env を書き換えるだけ。コード変更なし

```
OAUTH2_CUSTOM1_NAME='auth0'  
OAUTH2_CUSTOM1_CLIENT_ID='xxx'  
OAUTH2_CUSTOM1_CLIENT_SECRET='xxx'  
OAUTH2_CUSTOM1_ISSUER_URL='https://your-tenant.auth0.com'
```

ストレージも .env で切り替え

```
# SQLite (開発・デモ用、セットアップ不要)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# PostgreSQL / MySQL / MariaDB
GENERIC_DATA_STORE_TYPE=postgres      # or mysql
GENERIC_DATA_STORE_URL='postgres://user:pass@localhost/mydb'

# Cache: in-memory または Redis
GENERIC_CACHE_STORE_TYPE=memory       # or redis
```

- コード変更不要

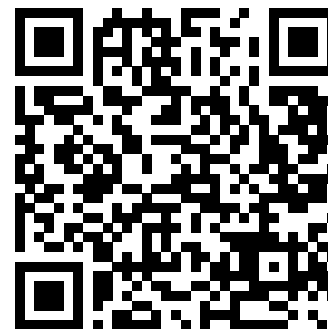
まとめ / Thank You!

oauth2-passkey

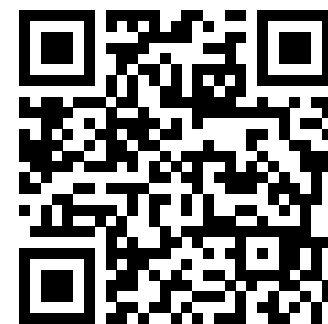
- Rust / Axum 向け認証ライブラリ
- crates.io 公開済み
- 複数 IdP + 複数 Passkey を **account linking**
- **Auth0 / Okta** などのOIDC IDP 対応

高橋 公俊

- フリーランス
- Rust 3 年目
- 一緒にスタートアップしませんか



GitHub



Contact